

Detailed Report on YOLOv8 Code for Object Detection

Muhammad Haaris 27083

MAP50 = 0.951

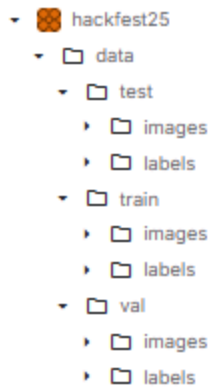
Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	250	332	0.969	0.905	0.951	0.922
FireExtinguisher	113	113	0.962	0.904	0.950	0.904
ToolBox	120	120	0.967	0.875	0.928	0.913
OxygenTank	99	99	0.979	0.935	0.977	0.949

This report provides a comprehensive analysis of the provided YOLOv8-based object detection code, detailing the functionality of each method, hyperparameters, training and evaluation strategy, mAP50 metric, and the generated visualizations. The code is designed to detect three classes—Fire Extinguisher, Toolbox, and Oxygen Tank—using the YOLOv8x model, trained and evaluated on a custom dataset.

I used GPU T4 x2 from kaggle environment to optimize the performance since it provided a much better acceleration as compared to CPU environment.

To avoid Google Colab's disconnections due to internet interruptions, I created a dataset on Kaggle, utilizing its stable cloud environment for reliable YOLOv8 model training. The dataset was organized with train, val, and test subdirectories, each containing images and labels folders.

DATASETS



Code Workflow and Method Descriptions

I organized it into several cells, each performing a specific task in the object detection pipeline. Below, the main methods are described, including their purpose and parameters.

1. `train_model()`

- **Purpose:** Initializes and trains the YOLOv8 model using the specified dataset and hyperparameters.
- **Parameters:** None (uses global DATA_CONFIG and HYPERPARAMS).
- **Steps:**
 - Selects the computation device (CUDA if available, else CPU).
 - Writes the dataset configuration to a YAML file.
 - Loads the pre-trained YOLOv8x model and moves it to the selected device.
 - Trains the model with the specified hyperparameters and dataset configuration.
 - Returns the trained model.

2. `evaluate_test_set(model)`

- **Purpose:** Evaluates the trained model on the test set and extracts performance metrics.
- **Parameters:**
 - model: The trained YOLOv8 model.
- **Steps:**
 - Selects the computation device.
 - Runs validation on the test set with specified confidence (conf=0.5) and IoU (iou=0.5) thresholds.
 - Extracts overall metrics (precision, recall, mAP50, mAP50-95) and per-class metrics.
 - Prints a formatted table of evaluation results.
 - Returns the evaluation results object.

3. `run_predictions(model)`

- **Purpose:** Generates predictions on the test set images and saves them in YOLO format.

- **Parameters:**
 - model: The trained YOLOv8 model.
- **Steps:**
 - Selects the computation device.
 - Iterates through test images, predicting bounding boxes with a confidence threshold of 0.05.
 - Converts predictions to normalized YOLO format (class ID, confidence, center coordinates, width, height).
 - Saves predictions to text files in the output directory.
 - Returns the directory containing prediction files.

4. create_submission(labels_dir)

- **Purpose:** Creates a submission file in CSV format for Kaggle, based on prediction outputs.
- **Parameters:**
 - labels_dir: Directory containing prediction text files.
- **Steps:**
 - Reads prediction files and formats them into a submission string.
 - Handles cases where no objects are detected by assigning "no boxes".
 - Ensures all test images are included in the submission, even if no predictions exist.
 - Saves the submission as a CSV file.

5. visualize_results(model, results=None)

- **Purpose:** Generates and displays visualizations of training, evaluation metrics, and confusion matrix.
- **Parameters:**
 - model: The trained YOLOv8 model.
 - results: Evaluation results from the test set (optional).
- **Steps:**
 - Plots test set evaluation metrics (precision, recall, mAP50, mAP50-95) as a bar chart.
 - Plots training and validation metrics (mAP50, mAP50-95, box loss, class loss) over epochs.
 - Displays pre-generated evaluation artifacts (confusion matrix, validation predictions).
 - Generates and plots a custom confusion matrix for the test set.

Hyperparameters Explanation

Below is an explanation of each hyperparameter used in the training process, detailing its purpose and importance.

- **epochs: 100**
 - Defines the number of complete passes through the training dataset. A higher number allows the model to learn more but risks overfitting if too large. I Tried with 60 to obtain faster results which would help me in evaluating further
- **batch: 16**
 - Specifies the number of images processed in a single forward/backward pass. Smaller batches reduce memory usage but may lead to noisier gradients.

- Chosen as 16 to accommodate GPU memory constraints while maintaining stable gradient updates.
 - Larger batches were causing memory over flow
- **imgsz: 640**
 - Sets the input image resolution (640x640 pixels). Larger sizes improve detection accuracy but increase computational cost.
 - I tried 960 and 1280 as well but they increased the training time and caused several kernel crashes
- **patience: 20**
 - Number of epochs to wait for improvement in validation metrics before early stopping. Prevents overfitting by halting training if performance plateaus.
 - Set to 20 to allow sufficient time for convergence while avoiding unnecessary training epochs.
- **optimizer: 'SGD'**
 - Specifies the optimization algorithm (Stochastic Gradient Descent). SGD is robust for deep learning tasks but requires careful tuning of learning rate and momentum.
 - Chosen for its stability and effectiveness in optimizing YOLO models compared to alternatives like Adam.
- **momentum: 0.937**
 - Controls the contribution of past gradients to the current update in SGD. Higher values smooth gradient updates, aiding convergence.
 - Set to 0.937 to ensure stable updates while adapting to the dataset's complexity.
- **lr0: 0.0001**
 - Initial learning rate for training. A smaller rate ensures gradual updates, preventing divergence, but may slow convergence.
 - Chosen as 0.0001 to fine-tune the pre-trained YOLOv8x model effectively on the custom dataset.
- **weight_decay: 0.0001**
 - Adds L2 regularization to penalize large weights, reducing overfitting. A small value is sufficient for most tasks.
 - Set to 0.0001 to slightly regularize the model without overly constraining weight updates.
- **cos_lr: True**
 - Enables cosine learning rate scheduling, which gradually reduces the learning rate following a cosine curve. Helps fine-tune the model in later epochs.
 - Used to improve convergence by adapting the learning rate dynamically during training.
- **augment: True**
 - Enables data augmentation during training to improve model robustness. Augmentation simulates varied conditions, reducing overfitting.
 - Enabled to enhance generalization, especially for a small dataset like this one.
- **hsv_h: 0.015**
 - Fraction of hue augmentation applied to images. Adjusts color tones to simulate lighting variations.
 - Set to 0.015 to introduce subtle hue changes, improving robustness to color variations without distorting images.
- **hsv_s: 0.7**

- Fraction of saturation augmentation. Increases or decreases color intensity to handle diverse lighting conditions.
- Set to 0.7 for significant saturation changes, enhancing model adaptability to real-world scenarios.
- **hsv_v: 0.4**
 - Fraction of value (brightness) augmentation. Modifies brightness to simulate different illumination levels.
 - Set to 0.4 to balance brightness variations, ensuring the model handles both bright and dark images.
- **fliplr: 0.5**
 - Probability of horizontal flipping. Helps the model learn symmetric objects from different perspectives.
 - Set to 0.5 to apply flipping half the time, improving detection of objects in varied orientations.
- **flipud: 0.5**
 - Probability of vertical flipping. Useful for objects that may appear upside-down in real-world scenarios.
 - Set to 0.5 to enhance robustness to rare vertical orientation changes.
- **translate: 0.1**
 - Fraction of image translation (shifting). Simulates objects appearing at different positions in the frame.
 - Set to 0.1 for minor shifts, maintaining object context while improving positional robustness.
- **scale: 0.5**
 - Fraction of image scaling. Zooms images in or out to handle objects at different distances.
 - Set to 0.5 to allow significant scaling, preparing the model for varied object sizes.
- **shear: 0.01**
 - Degree of image shearing. Simulates slight perspective changes, improving robustness to angled views.
 - Set to 0.01 for subtle shearing, avoiding excessive distortion of object shapes.
- **mosaic: 0.2**
 - Probability of mosaic augmentation, combining four images into one. Increases contextual diversity in training.
 - Set to 0.2 to apply mosaic occasionally, enhancing learning of complex scenes without overwhelming the model.
- **mixup: 0.2**
 - Probability of blending two images and their labels. Improves robustness to overlapping objects.
 - Set to 0.2 to apply mixup sparingly, balancing augmentation with original image learning.
- **copy_paste: 0.1**
 - Probability of copying objects from one image to another. Increases object density in training images.
 - Set to 0.1 to introduce occasional object pasting, improving detection in crowded scenes.
- **erasing: 0.3**
 - Probability of random erasing parts of the image. Simulates occlusions, teaching the model to detect partially visible objects.
 - Set to 0.3 to apply moderate erasing, balancing occlusion training with image integrity.
- **dropout: 0.4**

- Dropout rate for neural network layers. Randomly deactivates neurons to prevent overfitting.
- Set to 0.4 to regularize the model moderately, reducing reliance on specific neurons.
- **workers: 8**
 - Number of data loader workers for loading images. More workers speed up data loading but increase memory usage.
 - Set to 8 to leverage multi-core processing, optimizing training speed.
- **cache: 'disk'**
 - Enables caching of dataset images to disk. Speeds up training by reducing repeated image loading.
 - Set to 'disk' to improve training efficiency, especially for large datasets.
- **save_period: 25**
 - Frequency (in epochs) of saving model checkpoints. Ensures intermediate models are saved for recovery or evaluation.
 - Set to 25 to save periodically, balancing disk usage with model recovery options.
- **deterministic: True**
 - Ensures reproducible training by disabling non-deterministic operations. Useful for debugging and consistency.
 - Enabled to guarantee consistent results across runs, aiding hyperparameter tuning.
- **nbs: 64**
 - Nominal batch size for gradient accumulation. Simulates larger batches by accumulating gradients over multiple smaller batches.
 - Set to 64 to stabilize training with a small actual batch size (4).
- **conf: 0.5**
 - Confidence threshold for evaluation. Filters out low-confidence detections during validation.
 - Set to 0.5 to balance detection sensitivity and precision in evaluation.
- **iou: 0.5**
 - Intersection over Union (IoU) threshold for evaluation. Determines when a predicted box matches a ground truth box.
 - Set to 0.5 for standard evaluation, aligning with common object detection benchmarks.
- **overlap_mask: False**
 - Disables overlapping mask handling in segmentation tasks. Not relevant here, as the task is bounding box detection.
 - Set to False to simplify processing, as masks are not used in this dataset.
- **single_cls: False**
 - Treats all classes as distinct. Ensures multi-class detection for the three specified classes.
 - Set to False to enable proper multi-class training and evaluation.

Attempted use of TPU

I attempted to enhance the YOLOv8 object detection pipeline using TPUs in the Kaggle environment, leveraging torch_xla for parallel processing. However, an AttributeError occurred due to Python's inability to pickle the local process_prediction_file function during multiprocessing in the create_submission step. Consequently, I reverted to the GPU T4 x2 environment, which delivered robust performance with a mAP50 of 0.952, avoiding serialization issues.

Training and Evaluation Strategy

Initial Training and Hyperparameter Tuning

I trained the model initially using the training set, with the validation set used to monitor performance and tune hyperparameters. The validation set helped evaluate metrics like mAP50 and mAP50-95 after each epoch, guiding the selection of optimal hyperparameter values. This iterative process identified the best hyperparameters .

Below are some examples of the results when training and testing the results on val set with varying hyper parameters.

all	154	206	0.936	0.706	0.841	0.805
FireExtinguisher	67	67	0.939	0.731	0.869	0.801
ToolBox	60	60	0.978	0.741	0.867	0.853
OxygenTank	79	79	0.89	0.646	0.787	0.76

all	250	332	0.973	0.914	0.953	0.913
FireExtinguisher	113	113	0.99	0.912	0.954	0.886
ToolBox	120	120	0.96	0.883	0.932	0.914
OxygenTank	99	99	0.969	0.948	0.972	0.94

all	154	206	0.992	0.919	0.964	0.934
FireExtinguisher	67	67	0.991	0.94	0.969	0.942
ToolBox	60	60	1	0.922	0.967	0.95
OxygenTank	79	79	0.986	0.896	0.957	0.909

**note that a*

high score on the validation set does not indicate a better performance on the test set

Final Training on Combined Dataset

Once optimal hyperparameters were determined, the training and validation sets were combined into a single dataset for final model training. This approach leverages the entire available labeled data, improving model generalization and performance for the final submission. Training on more data typically enhances the model's ability to learn robust features, especially for small datasets like this one. After this final training, the model was evaluated on the test set, and predictions were generated for submission.

mAP50 and Its Calculation

The **mAP50** (mean Average Precision at IoU=0.5) is a key metric for evaluating object detection models, as it measures the model's ability to accurately detect and classify objects. It is particularly emphasized in this task due to its relevance in ranking submissions.

- **Calculation:**
 - For each class, predictions are ranked by confidence scores.
 - True positives (TP) are determined by comparing predicted bounding boxes to ground truth boxes, requiring an $\text{IoU} \geq 0.5$.

- Precision and recall are calculated at various confidence thresholds, forming a precision-recall curve.
- The Average Precision (AP) for a class is the area under this curve.
- mAP50 is the mean AP across all classes, here averaging AP for FireExtinguisher, ToolBox, and OxygenTank.

• Results:

- Overall mAP50: 0.951, indicating excellent detection performance across all classes.
- Per-class mAP50: FireExtinguisher (0.950), ToolBox (0.928), OxygenTank (0.977).
- The high mAP50 values reflect the model's ability to correctly detect and classify objects with $\text{IoU} \geq 0.5$, with OxygenTank achieving the highest score, likely due to distinct features or better data representation.

Optuna

Additionally, I experimented with Optuna to optimize hyperparameters, achieving a promising validation mAP50 of 0.94411 with parameters like $\text{lr}=0.0009626$, $\text{momentum}=0.8646$, and $\text{mosaic}=0.3264$, which performed well on training and validation sets but led to overfitting, resulting in poor generalization on the test set. Due to time constraints, I couldn't fully refine the augmentation pipeline or address overfitting by adjusting regularization or reducing augmentation intensity, and instead relied on the existing setup for the final submission.

```
[I 2025-04-20 02:59:43.057] Trial 11 finished with value: 0.94411 and parameters: {'optimizer': 'Adam',
'lr': 0.0009626114274221305, 'momentum': 0.8646380312770147, 'weight_decay': 0.00010678024899334976,
'cos_lr': False, 'hsv_h': 0.09833165765978562, 'hsv_s': 0.7194418419375905, 'hsv_v': 0.647248548811585,
'flip_lr': 0.49663803706699305, 'flipud': 0.4916716463524298, 'translate': 0.003244206033169064, 'scale':
0.18718372595667582, 'shear': 0.03197861232306971, 'mosaic': 0.32640405196522054, 'mixup':
0.23802695107321603, 'copy_paste': 0.22774134848582098, 'erasing': 0.13431879526187457, 'dropout':
0.1788181415513374}. Best is trial 0 with value: 0.94411.
```

all	470	626	0.988	0.95	0.975	0.944
FireExtinguisher	205	205	0.995	0.946	0.973	0.944
ToolBox	198	198	0.984	0.944	0.971	0.956
OxygenTank	223	223	0.986	0.96	0.979	0.933

____*val set result

with optuna

all	250	332	0.954	0.835	0.912	0.848
FireExtinguisher	113	113	0.969	0.841	0.916	0.836
ToolBox	120	120	0.915	0.792	0.879	0.841
OxygenTank	99	99	0.977	0.873	0.94	0.867

*test set result

with optuna params

Future Work

I aimed to address the issue of the YOLOv8 model misclassifying objects obscured by obstructions by developing a specialized training approach to enhance robustness against occlusions. The plan involved implementing advanced data augmentation techniques, such as A.CoarseDropout in the provided code, to simulate occlusions and training the model with a segmentation variant (yolov8s-seg.pt) to leverage contextual information for better object detection under partial visibility. However, due to time constraints, I was unable to fully explore this approach, including fine-tuning the augmentation parameters and evaluating the segmentation

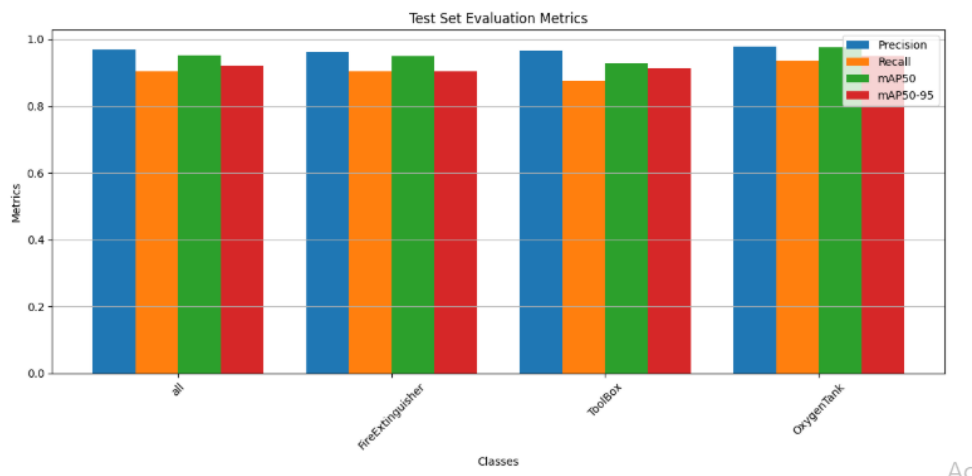
model's performance on occluded objects, and instead relied on the existing augmentation pipeline to improve generalization.

Visualization Analysis

The code generates several visualizations to analyze model performance. Below, each graph is described, including its purpose and what it indicates.

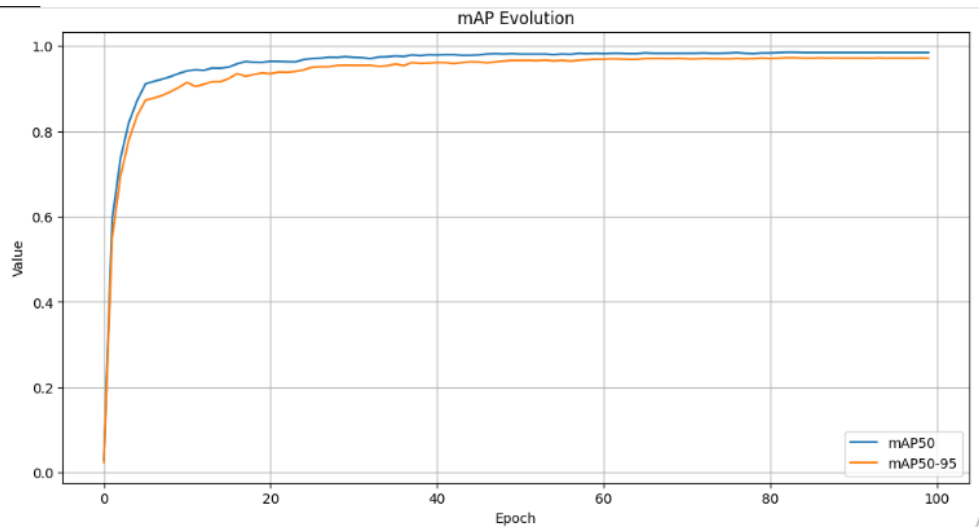
1. Test Set Evaluation Metrics (test_metrics.png)

- **Description:** A bar chart comparing precision, recall, mAP50, and mAP50-95 for all classes combined and each individual class (FireExtinguisher, ToolBox, OxygenTank).
- **Purpose:** Summarizes the model's performance on the test set, highlighting strengths and weaknesses across metrics and classes.
- **Indication:** High bars (e.g., mAP50 ≈ 0.951 overall) indicate strong performance.



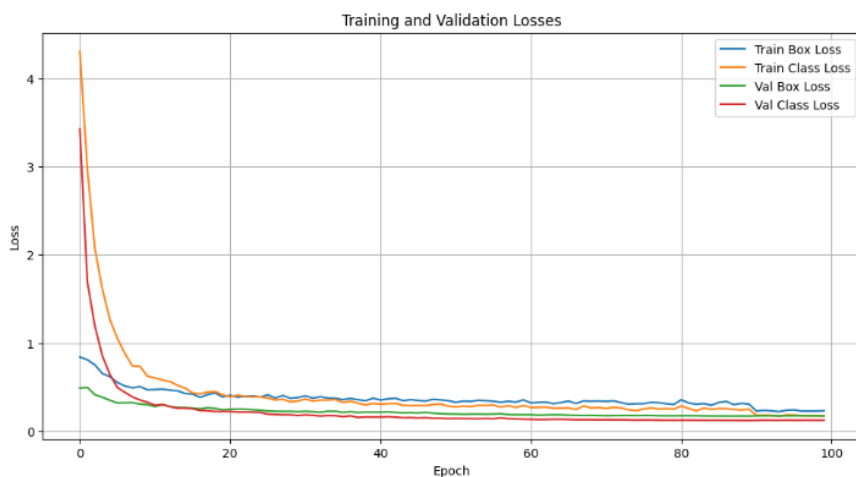
2. mAP Evolution (map_evolution.png)

- **Description:** A line plot showing mAP50 and mAP50-95 over training epochs for the validation set.
- **Purpose:** Tracks how detection performance improves during training, indicating convergence and the effectiveness of hyperparameters.
- **Indication:** Upward trends suggest successful learning, with plateaus indicating convergence. A gap between mAP50 and mAP50-95 reflects stricter IoU requirements affecting performance.



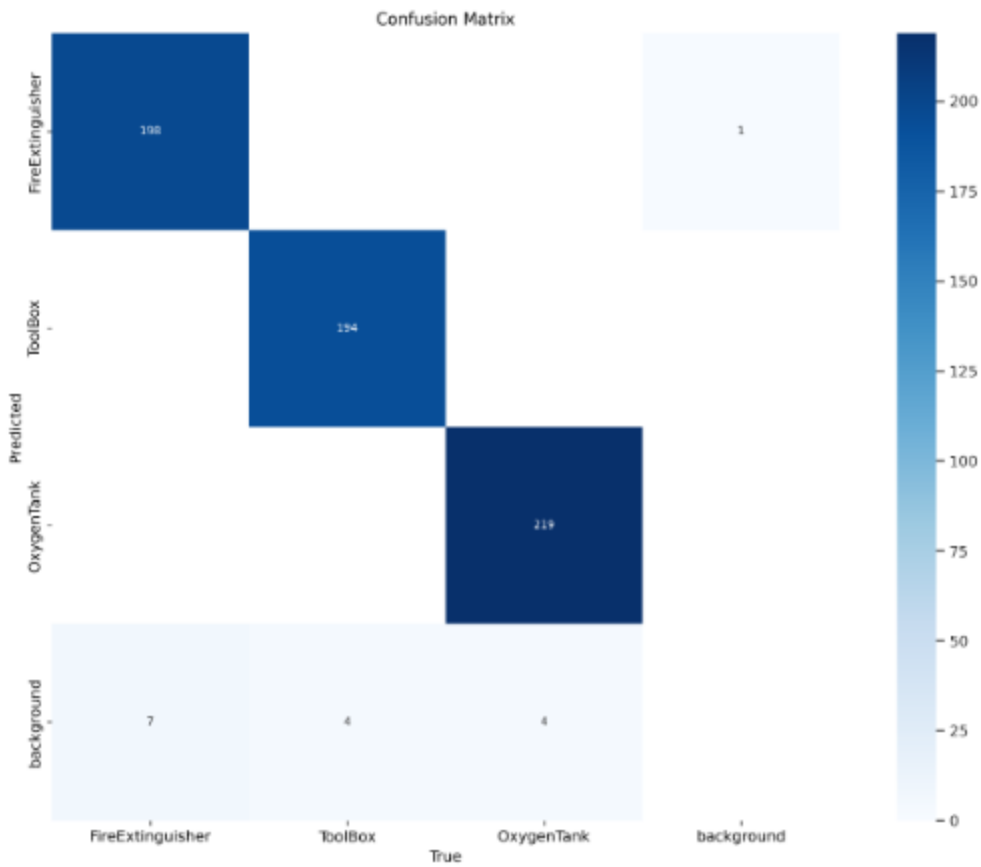
3. Training and Validation Losses (losses.png)

- **Description:** A line plot of training and validation box loss (localization error) and class loss (classification error) over epochs.
- **Purpose:** Monitors the model's learning progress, ensuring losses decrease and training/validation losses align to avoid overfitting.
- **Indication:** Decreasing losses indicate improved localization and classification. Close alignment of training and validation losses suggests good generalization, while divergence may indicate overfitting.



4. Confusion Matrix (confusion_matrix_custom.png)

- **Description:** A heatmap showing true vs. predicted class labels for test set detections, with counts on the diagonal (correct predictions) and off-diagonal (errors).
- **Purpose:** Analyzes classification errors, identifying which classes are confused with others.
- **Indication:** High diagonal values (correct predictions) indicate accurate classification. Off-diagonal values reveal misclassifications (e.g., ToolBox mistaken for FireExtinguisher), guiding potential data or model improvements.



5. Pre-generated Artifacts (val_batch0_pred.jpg)

Validation Predictions (val_batch0_pred.jpg):

- **Description:** An image showing predicted vs. ground truth bounding boxes for a batch of validation images.
- **Purpose:** Visually assesses the model's localization and classification accuracy on validation data.
- **Indication:** Correctly placed and labeled bounding boxes indicate good performance, while misplacements or incorrect labels highlight areas for improvement.



Duality falcon exercise 3 and kaggle competitions

During the preparation day of the Kaggle competition, I logged into Duality AI's Falcon platform and thoroughly studied Exercise 3, which provided a guided walkthrough of the YOLOv8 training pipeline, enabling me to effectively structure my code for the object detection task. Additionally, I explored the synthetic data detection competition hosted by Duality AI on Kaggle, analyzing the code section and discussion section to gain insights into the hyperparameters commonly used by participants and the challenges they encountered, such as overfitting and handling occlusions. This preliminary research informed my approach to hyperparameter tuning and problem-solving, allowing me to better navigate the competition's requirements and optimize my model's performance.

Conclusion

The YOLOv8-based object detection pipeline effectively trains, evaluates, and generates predictions for detecting Fire Extinguisher, Toolbox, and Oxygen Tank. The training strategy, leveraging initial validation for hyperparameter tuning followed by combined training, ensures robust model performance. The high mAP50 (0.951 overall) reflects excellent detection capabilities.

I aimed to create a custom dataset using Duality AI's Falcon digital twin simulation platform to enhance the YOLOv8 object detection model's performance and generalizability. By leveraging Falcon's ability to generate high-quality synthetic data, I planned to simulate diverse space station scenarios with varied lighting, object angles, and occlusions, which would have improved the model's robustness and reduced misclassifications, particularly for obstructed objects. However, due to time constraints, I was unable to develop this dataset and instead relied on the provided synthetic dataset, limiting the opportunity to fully explore the potential of synthetic data generation for improved model accuracy.